

13

Integrating a Vue.js Application with ASP.NET Core

Now, this is the time to make the Vue.js application talk to the REST API. In this chapter, you will integrate the Vue.js frontend application with the ASP.NET Core 5 backend application by installing a NuGet package and adding some code. You will also add a CORS policy in the backend to allow web apps from other domains to send requests to your backend.

We will cover the following topics:

- Putting the ASP.NET Core Web API and a Vue.js app together as a single unit
- Introducing **Cross-Origin Resource Sharing** or **CORS**
- Enabling a CORS policy in ASP.NET Core

Technical requirements

Here is what you need to complete this chapter:

- **Visual Studio 2019 IDE:** <https://visualstudio.microsoft.com/vs/>
- **Visual Studio for Mac IDE:** <https://visualstudio.microsoft.com/vs/mac/>
- **Rider IDE:** <https://www.jetbrains.com/rider/>
- **Visual Studio Code:** <https://code.visualstudio.com/>
- **Postman:** <https://www.postman.com/>

The finished code repo can be found here: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter13/>.

Putting ASP.NET Core Web API and a Vue.js app together as a single unit

The integration that we will do in this section is not required for Vue.js to send a request to the ASP.NET Core but will give us some advantages. Our goal here is to host the backend and the frontend projects in a single app project. In return, the application can be published or built as a single unit.

There are official web application project templates. You may call them boilerplates in .NET 5 that scaffold an Angular application with an ASP.NET Core Web API and a React application with an ASP.NET Core Web API. You can check out the two project templates at the following links:

- **Use the Angular project template with ASP.NET Core:** <https://docs.microsoft.com/en-us/aspnet/core/client-side/spa/angular?view=aspnetcore-5.0&tabs=visual-studio>
- **Use the React project template with ASP.NET Core:** <https://docs.microsoft.com/en-us/aspnet/core/client-side/spa/react?view=aspnetcore-5.0&tabs=visual-studio>

The boilerplates above are a convenient starting point for ASP.NET Core apps using React or Angular. The project can be quickly shipped as a single app to Azure App Service from your local machine from a Git repository such as GitHub or Azure DevOps.

Unfortunately, there is no official Vue.js with ASP.NET Core Web API boilerplate that exists. However, we can convert our ASP.NET Core API and Vue.js app into one project by adding some extra code in `Startup.cs`, updating `Travel.WebApi.csproj`, and installing a NuGet project. Are you excited? Let's start:

1. The first step is to install `VueCliMiddleware` in the `Travel.WebApi` project. The NuGet package is only needed in the `src | presentation | Travel.WebApi` project, so there's no need to install it in other projects.

`VueCliMiddleware` will allow us to build a Vue.js SPA on ASP.NET MVC Core using Quasar CLI or Vue CLI. Check out the nuGet package at this link and its GitHub repository: <https://www.nuget.org/packages/VueCliMiddleware> and <https://github.com/EEParker/aspnetcore-vueclimiddleware>.

2. The next step is to update the `Travel.WebApi.csproj` file. We are going to add some code below the last `ItemGroup` that in the file, which is this code:

```
<ItemGroup>
  <Folder Include="Extensions\" />
</ItemGroup>
```

3. We are going to add the following code to the `</ItemGroup>` closing tag. So make some space, hitting the *Enter* key several times, and then write the following code:

```
<!-- Below is for Single Page Application. -->
<!-- Please wait for compilation to finish before
going to https://localhost:5001 -->
<PropertyGroup>
  <!-- Typescript/Javascript Client Configuration
  -->
  <SpaRoot>..\vue-app</SpaRoot>
  <DefaultItemExcludes>$(DefaultItemExcludes);
  $(SpaRoot)node_modules\**</DefaultItemExcludes>
  <!-- Set this to true if you enable server-side
  prerendering -->
  <BuildServerSideRenderer>>false
  </BuildServerSideRenderer>
  <IsPackable>>false</IsPackable>
</PropertyGroup>
```

The newly added code tells the application that the directory of the SPA is in `vue-app`.

Also add the following code:

```
<ItemGroup>
  <!-- Don't publish the SPA source files, but do
        show them in the project files list -->
  <Content Remove="$(SpaRoot)**" />
  <None Remove="$(SpaRoot)**" />
  <None Include="$(SpaRoot)**"
        Exclude="$(SpaRoot)node_modules\**" />
</ItemGroup>
<Target Name="DebugEnsureNodeEnv"
  BeforeTargets="Build" Condition="
    '$(Configuration)' == 'Debug' And !Exists(
      '$(SpaRoot)node_modules') ">
  <!-- Ensure Node.js is installed -->
  <Exec Command="node --version"
    ContinueOnError="true">
    <Output TaskParameter="ExitCode"
      PropertyName="ErrorCode" />
  </Exec>
  <Error Condition="'$(ErrorCode)' != '0'"
    Text="Node.js is required to build and
    run this project. To continue, please install
    Node.js from https://nodejs.org/, and then
    restart your command prompt or IDE." />
  <Message Importance="high" Text="Restoring
    dependencies using 'npm'. This may take several
    minutes..." />
  <Exec WorkingDirectory="$(SpaRoot)" Command="
    "npm install" />
</Target>
```

The preceding code adds the `Spa Root` folder, which is `vue-app`, to the project.

4. Then add the final block of code we need in `Travel.WebApi.csproj`, like so:

```
<Target Name="PublishRunWebpack"
  AfterTargets="ComputeFilesToPublish">
  <!-- As part of publishing, ensure the JS
    resources are freshly built in production mode
  -->
  <Exec WorkingDirectory="$(SpaRoot)" Command=
    "npm install" />
  <Exec WorkingDirectory="$(SpaRoot)" Command=
    "npm run build" />
  <!-- Include the newly-built files in the publish
    output -->
  <ItemGroup>
    <DistFiles Include="$(SpaRoot)dist\**;"
      $(SpaRoot)dist-server\**" />
    <DistFiles Include="$(SpaRoot)
      node_modules\**"
      Condition="'$(BuildServerSideRenderer)' ==
        'true'" />
    <ResolvedFileToPublish Include="@ (DistFiles
      ->'%(FullPath)') " Exclude="
      @ (ResolvedFileToPublish) ">
    <RelativePath>%(DistFiles.Identity)
    </RelativePath>
    <CopyToPublishDirectory>PreserveNewest
    </CopyToPublishDirectory>
    <ExcludeFromSingleFile>true
    </ExcludeFromSingleFile>
  </ResolvedFileToPublish>
</ItemGroup>
</Target>
```

The added code is for running `npm run build` in the Vue.js root directory, which includes the newly-built files in the publish output.

That's all for `csproj` file. Now let's go to the next step.

5. Now go to `Startup.cs` and update the `ConfigureServices` method. Add the following code at the end of the services:

```
services.AddSpaStaticFiles(configuration =>
{
    configuration.RootPath = "../vue-app/dist";
});
```

The preceding code should be below `services.AddTransient<IConfigureOptions>`. The code adds a SPA static files service in the application.

6. Next, we update the middleware block, which is the `Configure` method. Add the following code under the `if (env.IsDeveloper())` block:

```
app.UseStaticFiles();
if (!env.IsDevelopment())
{
    app.UseSpaStaticFiles();
}
```

We are adding `UseSpaStaticFiles` in the middleware pipeline.

7. Now let's add `UseSpa`, another middleware, like so:

```
app.UseSpa(spa =>
{
    spa.Options.SourcePath = "../vue-app";

    if (env.IsDevelopment())
    {
        spa.UseVueCli(npmScript: "serve");
    }
});
```

The code adds the source path to `UseSpa` and also runs `Vue CLI serve` in production.

You might need to import `VueCliMiddleware` and `SpaServices` manually if your IDEs are not helping you:

```
using Microsoft.AspNetCore.SpaServices;
using VueCliMiddleware;
```

The preceding code is the namespaces that you will need to create a Vue CLI proxy mapping. `Travel.WebApi` and `vue-app` are inside the `presentation` directory, as shown in the following screenshot:

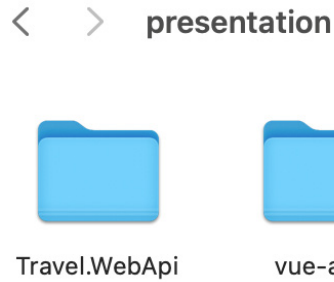


Figure 13.1 – The presentation directory

The two folders that you see in *Figure 13.1* are also what you will see in Visual Studio Code:

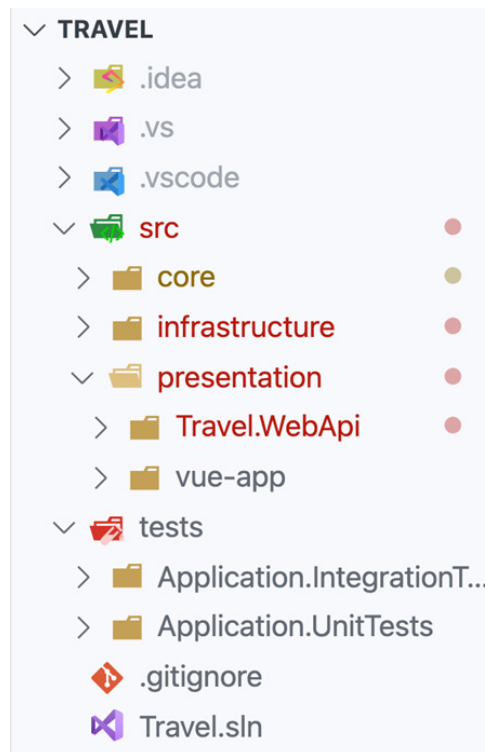


Figure 13.2 – VS Code folder arrangement

The VS Code folder arrangement in *Figure 13.2* also shows that `Travel.WebApi` and `vue-app` are on the same level in the directories. However, it is different in IDEs such as JetBrains' Rider, Visual Studio for Mac, and Visual Studio 2019, as shown in the following screenshot:

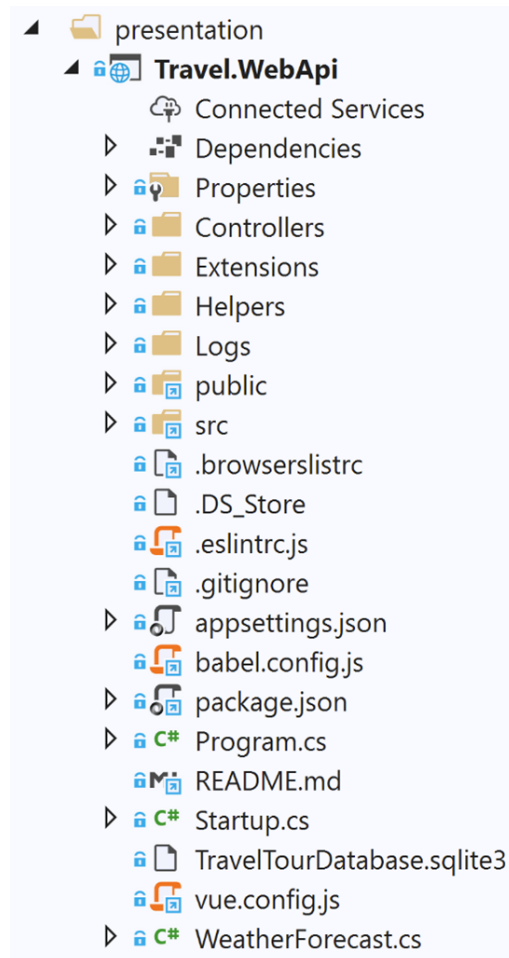
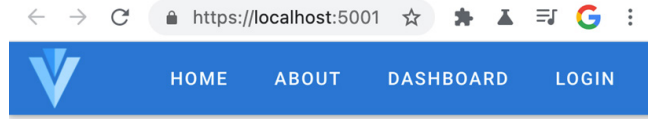


Figure 13.3 – Vue.js files and folders in the `Travel.WebApi` directory

IDEs are smart enough to know the SPA folders and files and expose them in the Solution Explorer's interface.

8. Now let's run the application by hitting your **IDE debug** button or running the `dotnet run` command inside the `Travel.WebApi` project. Then go to `https://localhost:5001` to see the Vue.js app as shown in the following screenshot:



Welcome to Travel Tours 🌍 🧳 ✈️

Figure 13.4 – Vue.js running on port 5001

Accept any popup from the browser that tells you the SSL certificate of the localhost. localhost:5001 is proxying port 8080 of the Vue.js CLI. Start editing the **Welcome to Travel Tours** message in your code base and see that Hot Module Replacement is still darn fast. You can see the changes immediately after saving any changes.

9. Now check out the Swagger UI to see that it is also running on the same port:

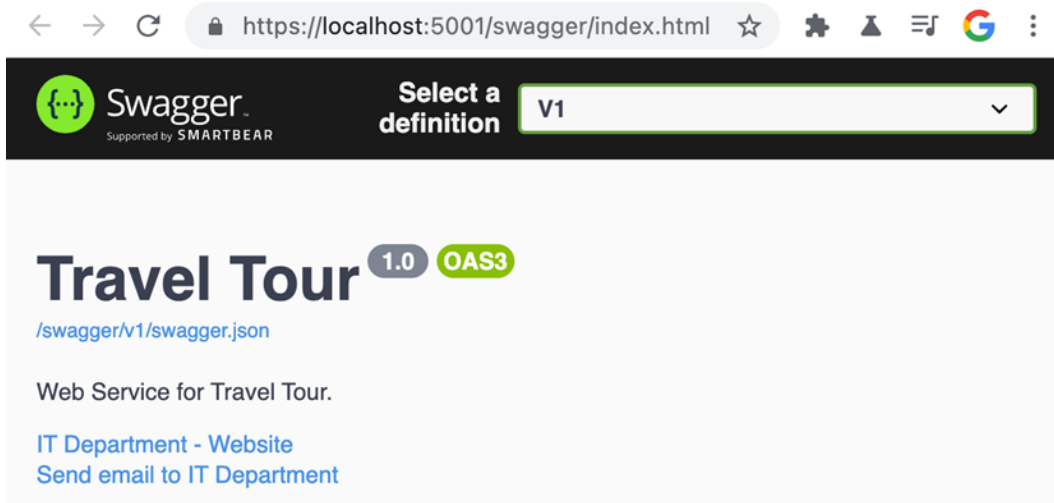


Figure 13.5 – Swagger UI on port 5001

Figure 13.5 shows that Swagger UI is still working on port 5001.

Now we can ship the application as a single unit. What if you have another SPA that lives in another domain and wants to send requests to your ASP.NET Core?

It is not possible right now, and that is what we will fix in the next section of this chapter.

Introducing Cross-Origin Resource Sharing or CORS

Before we discuss CORS policy and cross-origin resource sharing, try to send a POST request to version 2 of the `WeatherForecast` endpoint. See if you can still retrieve some of the JSON response of the `WeatherForecast` controller using Postman as shown in the following screenshot:

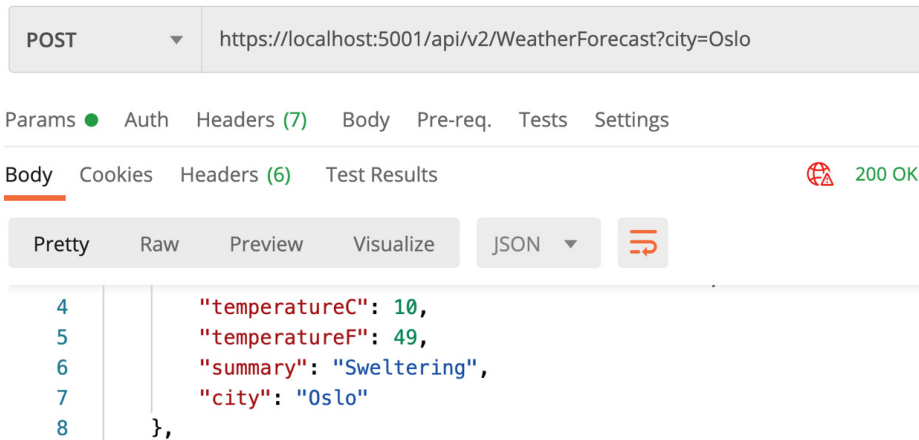


Figure 13.6 – Sending a POST request to `WeatherForecast` using Postman

Figure 13.6 means that the endpoint is still working correctly, but it would not work in another SPA.

I created a React application that runs on port 3000 to see if it can fetch the JSON objects from the `WeatherForecast` controller. No auth is required in the request, but the React application logs errors in the console; see Figure 13.7:

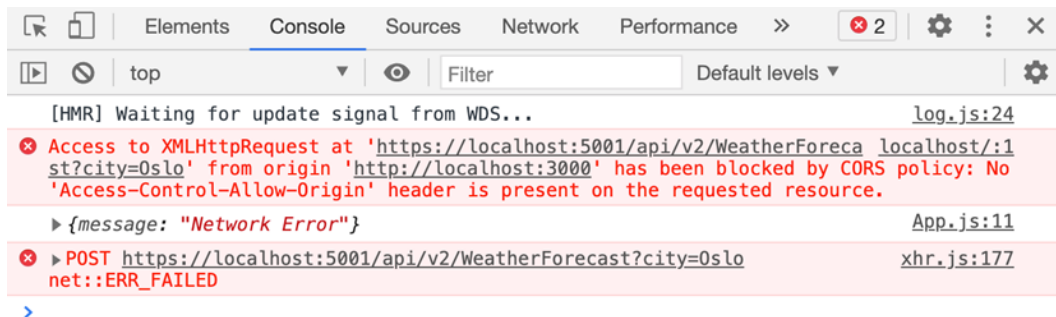


Figure 13.7 – Blocked by CORS policy

The error in *Figure 13.7* says that access to XMLHttpRequest at the endpoint from localhost:3000 has been blocked by CORS policy. The No Access-Control-Allow-Origin header is present on the requested resource. If you want to see it for yourself, you can run the React application you can get from the GitHub repo's chapter 13. Use the react-app folder by running `npm install` then `npm run start`, which will open your browser on localhost 3000.

So what is happening here? How can we solve this?

CORS stands for **Cross-Origin Resource Sharing**. It's a pretty good security concept. If both a client app and the APIs come from the same server, like a traditional web, sending HTTP requests will just succeed. The server processes a request because the request is trying to access resources on the same server.

However, sometimes even the localhosts are considered as a different origin. If origins are not the same, in React using port 3000 as an example, React from port 3000 sends requests to the server to get a resource, but the browser will fail the requests of React.

Since we are building RESTful APIs, we want to allow this access because Web APIs are meant to be consumed by different browser applications. I explicitly said browser applications because CORS restrictions are not applied in mobile apps, so mobile apps don't need to send a pre-flight request. A preflight request is a quick, small request that the browser sends to the resource server. The pre-flight request checks whether the CORS protocol is understood and the server has policies for processing methods and headers.

Now we know what CORS is, let's update our backend to enable CORS policy configuration in the next section.

Enabling a CORS policy in ASP.NET Core

Our goal in this section is to use an interface with CORS policy and configure it to allow any SPAs to send requests to our ASP.NET Core Web APIs.

So let's go to the `Startup.cs` file of the `Travel.WebApi` project and update the `ConfigureServices` method by adding the following code under the `AddSpaStaticFiles` method:

```
services.AddCors();
```

The `services.AddCors` method will be the last method we are calling inside `ConfigureServices`. This service is an extension method for setting up CORS services.

We are not yet done here; we still need to update our middleware. So, go to the `Configure` method and add the following code under `app.UseSpaStaticFiles`:

```
app.UseCors(b =>
{
    b.AllowAnyOrigin();
    b.AllowAnyHeader();
    b.AllowAnyMethod();
});
```

The `UseCors` method adds a CORS middleware to our web application's pipeline. This middleware will allow cross-domain requests in our application from now on.

Now let's give it a go and see if we can directly send a request coming from a different port.

You can run the React application by running `npm run start` in its root folder. You should see the same results as shown in the following screenshot:

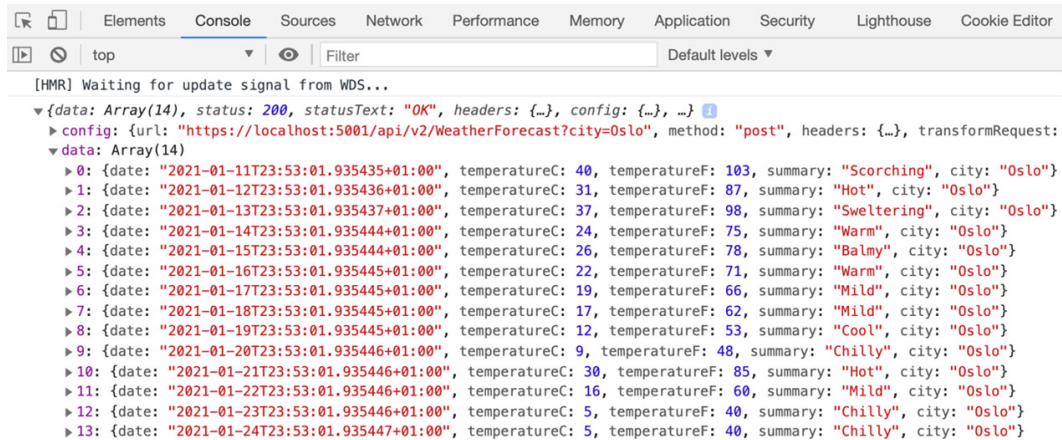


Figure 13.8 – React from port 3000 getting a 200 OK response

Figure 13.8 shows that our CORS policy configuration is working, and it is not blocking the request from React coming from `localhost:3000`.

Important

You can learn more about CORS policy at <https://auth0.com/docs/applications/set-up-cors>.

Let's now summarize what you have learned.

Summary

You have arrived at the end of this chapter; you have learned how to include the Vue.js app in the ASP.NET Core Web API project to run and ship them together as one. You also know how **Cross-Origin Resource Sharing** or **CORS** works. In a nutshell, it's a security feature of the browser that blocks requests from different domains or origins. You have learned how to enable and configure a CORS policy in ASP.NET Core to allow incoming requests from other domains or origins.

In the next chapter, we will start sending GET requests to our ASP.NET Core Web API and use Vuex for managing the state of our Vue.js app. It will be a significant and big chapter, so take a break first and come back after 40 to 60 minutes.